CS4287 - Neural Computing

U-Net Semantic Image Segmentation on Cityscapes

Assignment 1: Convolutional Neural Networks (CNNs)

Submitted by: Group 8

Igor Kochanski 23358459, Emily Domini 23362235, Nikolay Stoiakine 23380969

Professor: J.J. Collins
Submitted: 31st October 2026



1. Overview **3**
2. Dataset **3**
2.1 Visualisation of Key Attributes 4
2.2 Data Pre-Processing 6
3. Network Structure & Hyperparameters **8**
3.1 Weight initialisation 10
3.2 Activation Function 11
3.3 Residual Connections 12
3.4 Hyperparameters 13
4. Training and Validation **13**
4.1 Loss function 13
4.1.1 Weighted Cross-Entropy 13
4.1.2 Lovász-Softmax 14
4.2 Optimizer 14
4.2.1 Adaptive Moment Estimation with Weight Decay 14
4.2.2 Learning Rate Scheduler 14
4.3 Metrics 15
4.3.1 Loss 15
4.3.2 Pixel Accuracy 15
4.3.3 Mean Class Accuracy 15
4.3.4 Mean Class Intersection Over Union 16
4.4 Training Epochs 16
4.5 Cross Fold Validation 17
4.5.1 Data Split 17
4.5.2 Training Setup 18
4.5.3 Discussion 19
4.5.4 Limitations 20
5. Evaluation of Results **20**
5.1 Cross Fold Validation Results 20
5.2 Training Performance 21
5.3 Final Test Evaluation 22
6. Independent Experimental Work **26**
6.1 Training Experimentation 26
6.2 Model Architecture Experimentation 29
6.2.1 Baseline 29
6.2.2 Multi-Scale Dilated Bottleneck 30
6.2.3 Squeeze-and-Excitation Blocks 31
6.2.4 Residual Connections 31
7. Estimated Level of Difficulty **33**
8. Statement of Work **33**

9. Use of Generative AI.....33
10. References..... 36

1. Overview

Our project implemented a convolutional neural network based on **U-Net** from scratch for semantic segmentation on the **Cityscapes** dataset. The aim was to accurately segment an image into regions by assigning one of 19 class labels to every pixel. We explored the dataset for insights, experimented with training optimisation and added residual connections to deeper model layers to improve performance. Our final network consisted of 30 convolutional layers and 8,112,563 parameters in total, achieving a mean Intersection over Union (IoU) score of **0.5665**, pixel accuracy of **0.9204**, and mean class accuracy of **0.7408**.

The project source code, trained model weights and detailed results can be found on GitHub: https://github.com/BaguetteJet/u-net-cityscapes

2. Dataset

This project was based on the **Cityscapes Dataset** [1][2], which focuses on semantic understanding of urban street scenes. It consists of street images taken from 50 German, Swiss, and French cities during daytime, across several months. Each image comes with pixel-level annotations classifying objects and regions. All images are at **1024x2048 resolution** in RGB format saved as PNG.

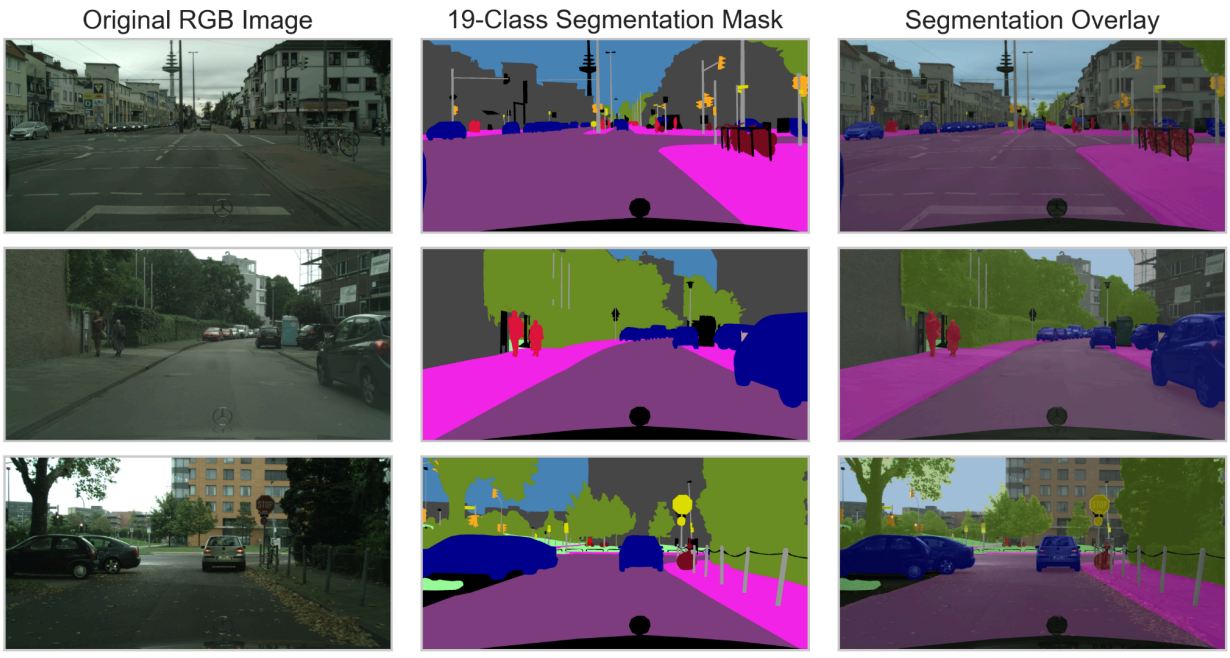


Figure 1 - Sample images from the dataset, their annotated masks, and the overlay of the two

The Cityscapes website provides left and right camera images, but annotations are only provided for the left side. There are 5,000 finely annotated images and 20,000 coarsely annotated images. For this project, the finely annotated left 8-bit image package was used.

1,525 of the 5,000 images are used for the official Cityscapes benchmark, with no public annotations available. This leaves **3,475 images** that are annotated and usable.

Images come with annotations for 34 classes, but only 19 classes are used for evaluation in the official Cityscapes benchmark. This project focuses on these **19 classes**, which include: roads, sidewalks, buildings, walls, fences, poles, traffic lights, traffic signs, vegetation, terrain, sky, pedestrians, riders, cars, trucks, buses, trains, motorcycles, and bicycles. The remaining classes were ignored during training and evaluation.

2.1 Visualisation of Key Attributes

Class pixel frequency and class image frequency are not evenly distributed. Classes such as roads, buildings, and vegetation dominate the scenes and cover over 75% of the total pixels in the dataset, while trains, buses or traffic lights rarely appear at all, occupying roughly 1% combined. This **class imbalance** introduces bias in the model towards the more densely represented classes, making it likely to overpredict these classes and underpredict the rarer classes.

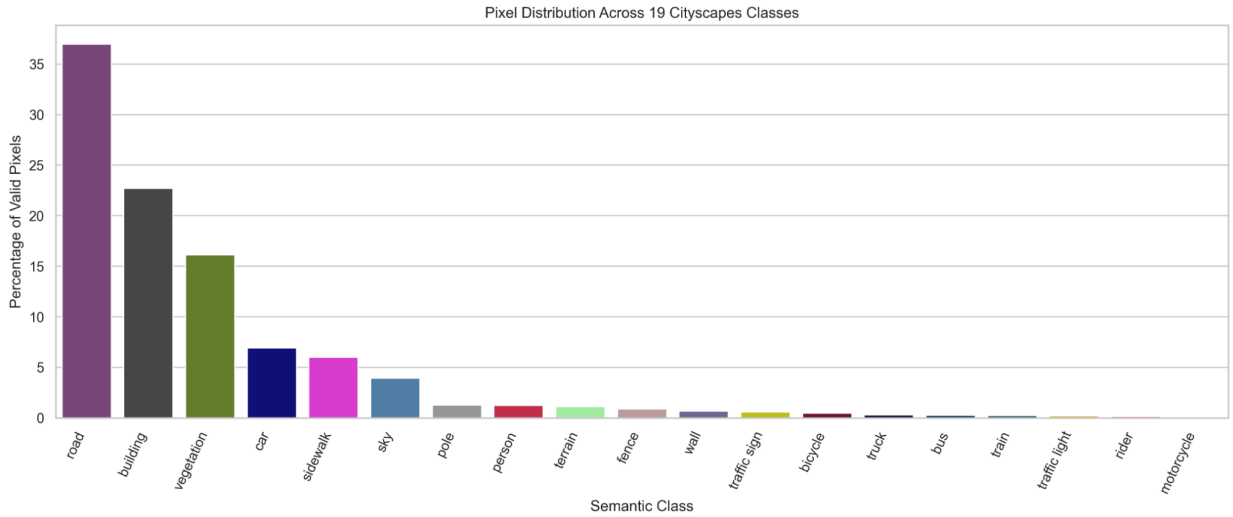


Figure 2 - Pixel distribution by class (colours adjusted to match the official Cityscapes palette)

In order to alleviate this class imbalance and ensure our model learns to predict each class as accurately as possible, we added weights to our loss function. The weights were derived from the pixel frequency calculated in Figure 2. This is discussed in more detail in section 4.1.

We examined the distribution of the 19 semantic classes across each city in the dataset, so that we could ensure our split would contain a balanced representation of each class. It's important that there aren't certain cities which have a much denser presence of a particular class, to avoid class-specific bias.

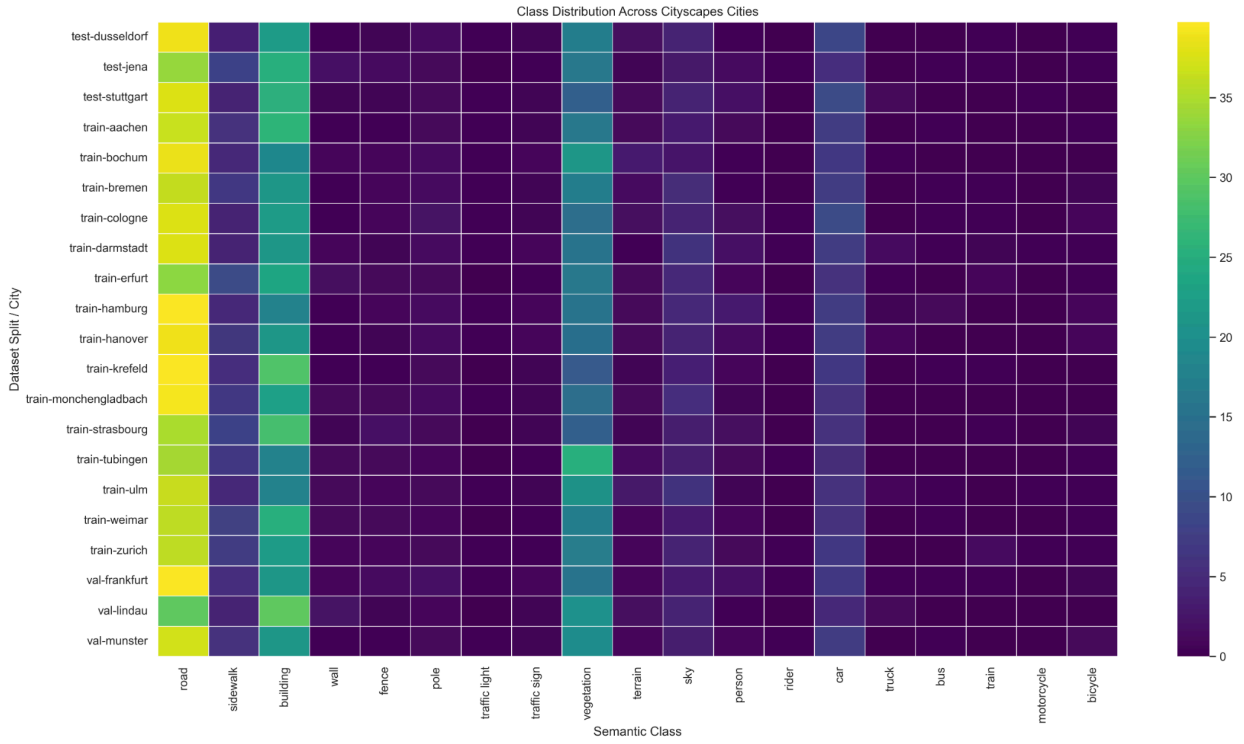


Figure 3 - Heatmap of class distribution by city

Figure 3 confirms very little variance in class distribution across each city. As each city then provides a fair and uniform representation of each class, it would be acceptable to divide our training, validation, and test splits by city. Figure 3 also evidences the dominance of classes such as road, building, and vegetation across each city, emphasising the need for a weighted loss function.

We then analysed the class diversity, brightness, and contrast of images per city. Brightness and contrast were fairly uniform across cities, with some deviation as expected. The level of variance in urban street scene datasets is usually due to the weather or time of day when the images were captured, and to the colour palette of the architecture in different cities. The variance is not outside of the average range and we did not identify any major outliers. As for class diversity, only one city stood out. Lindau had a median of 8 classes per image, where the median across other cities all lay between 11 and 13.

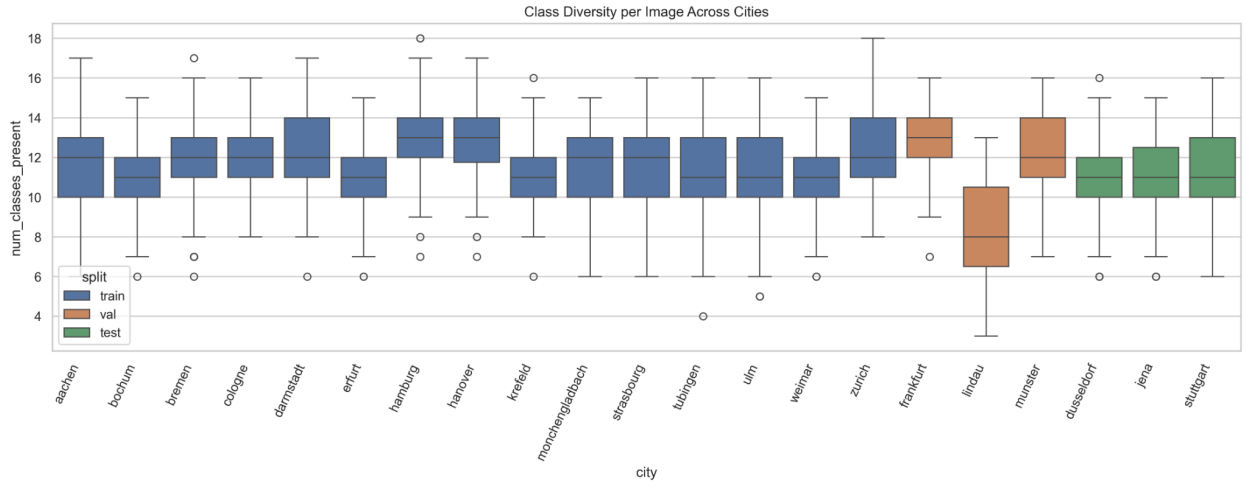


Figure 4 - Class diversity per image across all cities

This is generally a sign of a different scene composition and a different type of urban environment than what is typically seen in the other cities. We decided to keep Lindau in our split to preserve the range of visual conditions represented in the dataset.

2.2 Data Pre-Processing

The data visualisation step confirmed an even distribution of classes across each city in the Cityscapes dataset. As the official Cityscapes test set does not have publicly available annotations, we set aside three cities from the training and validation set to be used for local testing. We decided on a **70/15/15 split**, dividing our **21 cities** into 15/3/3 for training, validation, and testing respectively. Out of the 3,475 total images, this designated 2,439 images for training, 500 for validation, and 536 for testing.

Training set: 2,439
Aachen, Bochum, Bremen, Cologne, Darmstadt, Erfurt, Hamburg, Hanover, Krefeld, Monchengladbach, Strasbourg, Tübingen, Ulm, Weimar, and Zurich

Validation set: 500
Frankfurt, Lindau, and Munster

Test set: 536
Dusseldorf, Jena, and Stuttgart

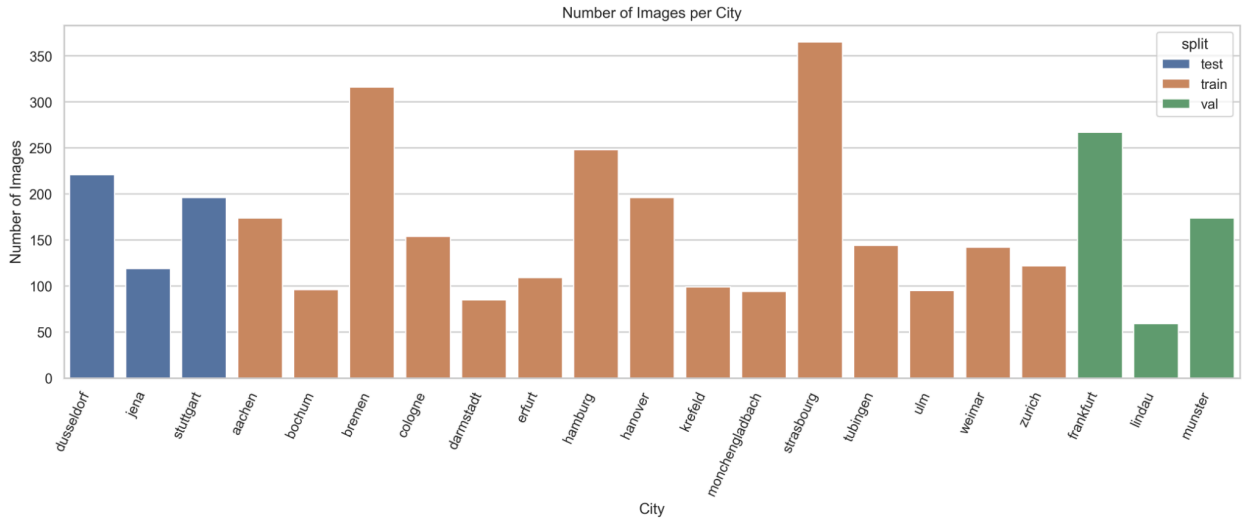


Figure 5 - Number of images per city, coloured by split

The distribution of images per city displays a significant variance, with a mean image count of 165.48, a median of 144, and a standard deviation of 80.68. Cities like Strasbourg and Bremen have notably more images than cities like Bochum and Darmstadt. This means that the cities with more images will have a greater influence on the model's learning. This could matter if the city with more images has a very different street and road structure than other cities, but Figure 3 rebuked this.



Figure 6 - Heatmap of class distribution across splits

Our splits exhibit a balanced distribution of classes, evident in Figure 6.

All image masks were converted to contain only the selected 19 class labels. The remaining classes were reassigned to 255, which was the ignored label during training and evaluation. This was the recommended official Cityscapes method.

Training set images were further **augmented** during training. Each image was randomly cropped, scaled and optionally horizontally flipped. The scale or zoom was chosen uniformly

between 0.5 and 2, where 0.5 would be the whole image. The position of the zoomed window was randomised. Each image was resized to 512x1024 to allow for a larger batch size.

Any transformation done on an image was also done to the corresponding image mask. Images were resampled bilinearly, while the masks used nearest-neighbour interpolation, so that class IDs wouldn't blend into invalid values. At the end, images would randomly horizontally flip with 50% probability. Validation and test set images remained unaltered.

3. Network Structure & Hyperparameters

The **U-Net** architecture was established by Olaf Ronneberger, Philipp Fischer, and Thomas Brox from the University of Freiburg, Germany in 2015 [3]. U-Net is a u-shaped **convolutional neural network** (CNN) [5] consisting of **contracting** (downsampling) operations which capture context followed by a symmetrical path of **expansive** (upsampling) operations for precise localisation. The two paths are linked by **skip connections**, which carry high resolution data from the encoder (downsampling) path to its corresponding decoder (upsampling) block. The U-Net was initially developed for image segmentation in the biomedical field, but has since become a foundational architecture in computer vision and a core denoising component in modern generative AI and diffusion models.

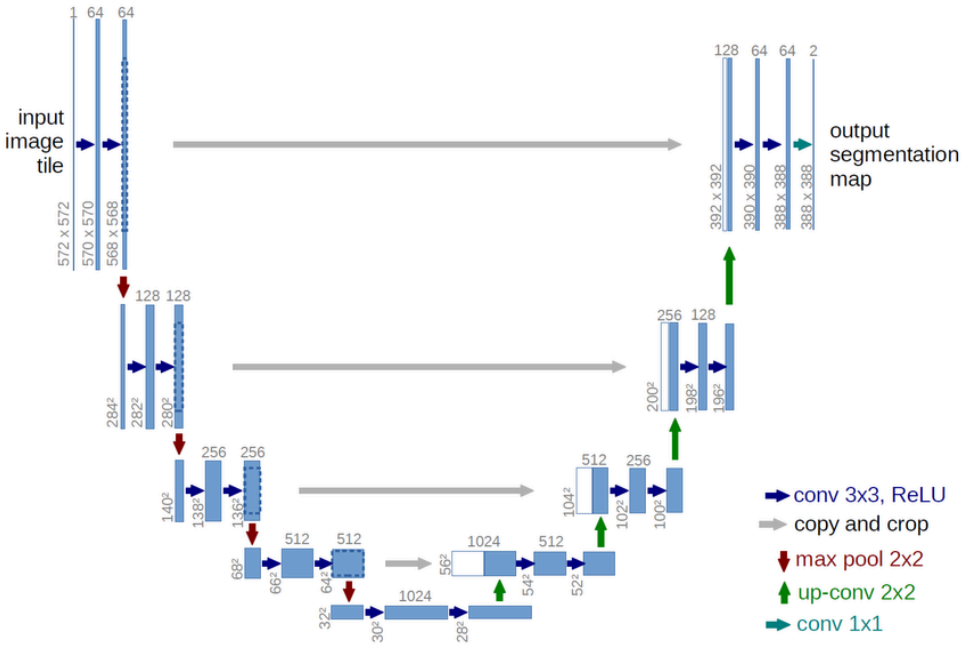


Figure 7 - The original U-Net architecture [3]

The motivation behind the original U-Net was the very small supply of data for biomedical imagery and a need for more accurate image segmentation in this field. Ronneberger et. al. built upon the “fully convolutional network” by Long, Shelhamer, and Darrell (2015) [9], modifying it to work well with very few training images and to produce more precise segmentation. The

approach was highly successful for identifying, labeling, and separating biological structures in electron microscopy (EM) images. Their U-Net achieved a warping error of 0.0003529 and a rand-error of 0.0382, beating the 2012 winners of the EM segmentation challenge at the International Symposium on Biomedical Imaging (ISBI).

We chose a U-Net architecture for this project as it had not been covered in lectures, presenting an opportunity to explore a new and challenging architecture independently. Although U-Nets can support other image-based tasks, they are particularly well-suited to semantic segmentation due to their encoder-decoder architecture and skip connections. These characteristics made it a suitable choice for the Cityscapes dataset, where accurate pixel-level classification is required.

The base building blocks of our U-Net were **double convolution blocks**, each consisting of two sequential sets of a 3x3 convolution, a batch normalisation and a rectified linear unit (ReLU) activation. These blocks had an optional residual connection used only for deeper layers to improve feature propagation. This is explained in more detail in section 3.3. Convolutions were padded to preserve spatial size, unlike the original U-Net.

The **decoder** first used a double convolution that converted the 3 channel (RGB) input into 32 feature channels. Then four "**down**" blocks, each consisting of a 2x2 max pooling layer of stride 2, and a double convolution. Each down block halved the feature map resolution and doubled the feature channels. The final of four down blocks was the deepest layer of our model with 512 feature channels, and is known as the "**bottleneck**".

Next, the model moved onto the **encoder**, which was made of four "**up**" blocks. Each up block consisted of a 2x2 transposed convolution, skip connection concatenation with the corresponding down block output at the current depth, and a double convolution. Up blocks doubled the feature map resolution and halved the feature channels. This was followed by a final 1x1 convolution layer to convert the 32 feature channels into the 19 output classes.

Batch normalisation (BN) was used to standardize between layers across mini-batches. Normalisation was applied before the activation function (ReLU) so that it captured negative values when calculating mean and variance before the activation function made negatives zero. Group normalisation was considered as an alternative, but ultimately it was not used as during experimentation it consumed substantially more memory and made training difficult.

Transposed convolutions (up conv) in the encoder were used to upscale the image by subsampling and reducing channels in a single step. The upsampling kernel is learned, adding trainable parameters. Bilinear interpolation is an alternative method, which is computationally simpler, but does not reduce feature channels. The extra channel tensors consume more memory than the extra parameters from transposed convolutions. Transposed convolutions were selected over bilinear interpolation in order to reduce memory usage during training.

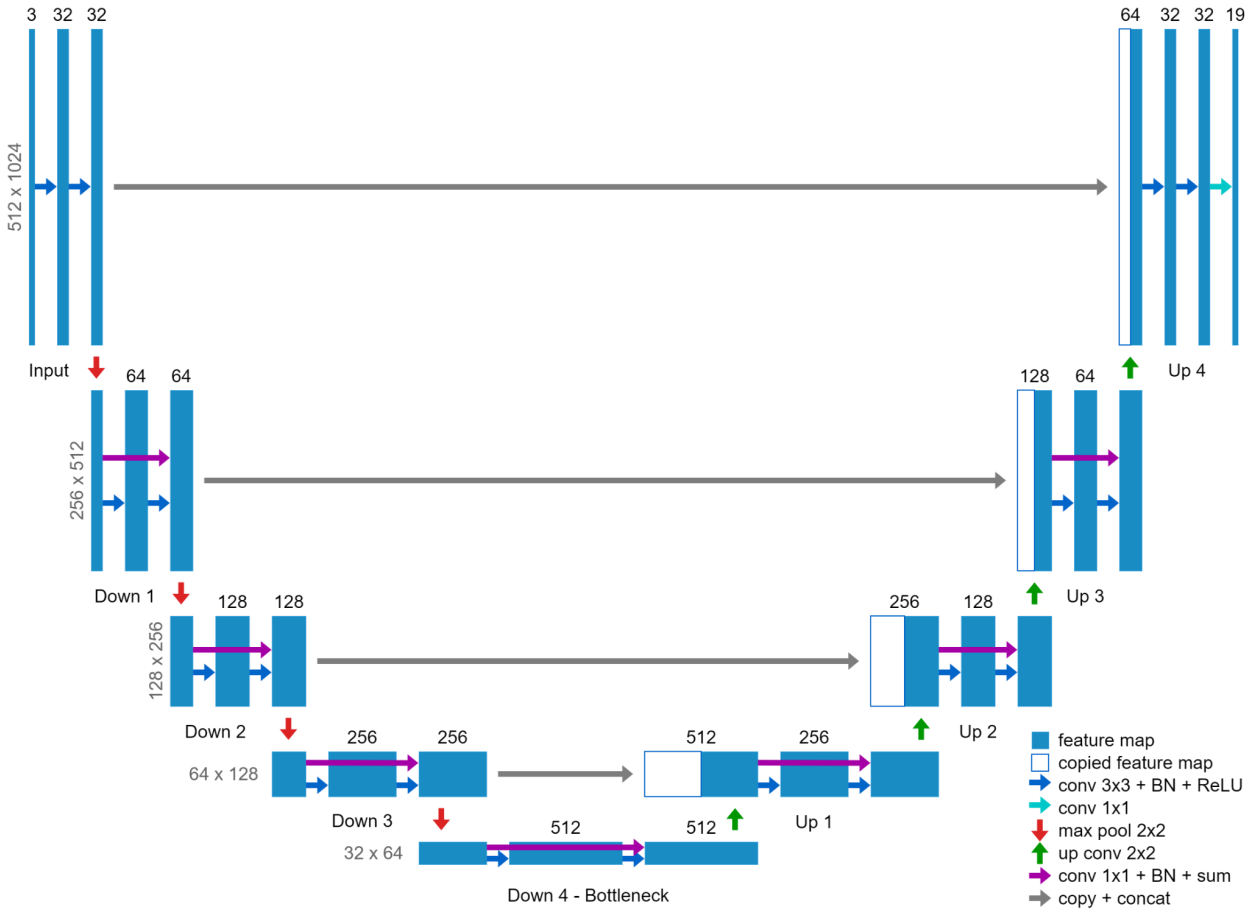


Figure 8 - V2.0.6 Our final convolutional neural network architecture based on U-Net with added residual connections. Dimensions provided for 512x1024 input crop size.

Our network had a total of **30 convolutional layers**, with no fully-connected layers. There were a total of **8,112,563 parameters**, all of which were trainable.

3.1 Weight initialisation

Weight initialisation influences the magnitude of activations and gradients at the model's instantiation, and affects training stability and convergence. We did not explicitly initialise weights for our network layers, so **PyTorch's default parameter initialisation** was used.

Convolutional and transposed-convolutional weights were automatically initialised using Kaiming uniform initialisation with **a =** $\sqrt{5}$, where 'a' represents the negative slope parameter used in the gain calculation for a Leaky ReLU activation. However, Kaiming initialisation is perfectly compatible with the standard ReLU activation, which was our activation function of choice.

Batch normalisation layers were initialised with learnable scaling parameters set to 1 and shifting bias parameters set to 0. This initially allowed the normalisation layers to operate without additional learned scaling or shifting, and to be adapted during training.

The residual connections did not require a separate initialisation procedure, and instead were initialised with the PyTorch default like all other convolutional layers. Altogether this provided randomly initialised model parameters which we later optimised during training.

3.2 Activation Function

The **ReLU activation function** was applied after every batch normalisation step in our model architecture. The ReLU is the most common activation function in deep learning, defined mathematically as:

$$f(x) = max(0, x)$$

It passes positive values through unchanged and sets all negative values to zero. Although it is piecewise linear, the combination across layers introduces non-linearity, allowing networks to learn complex patterns.

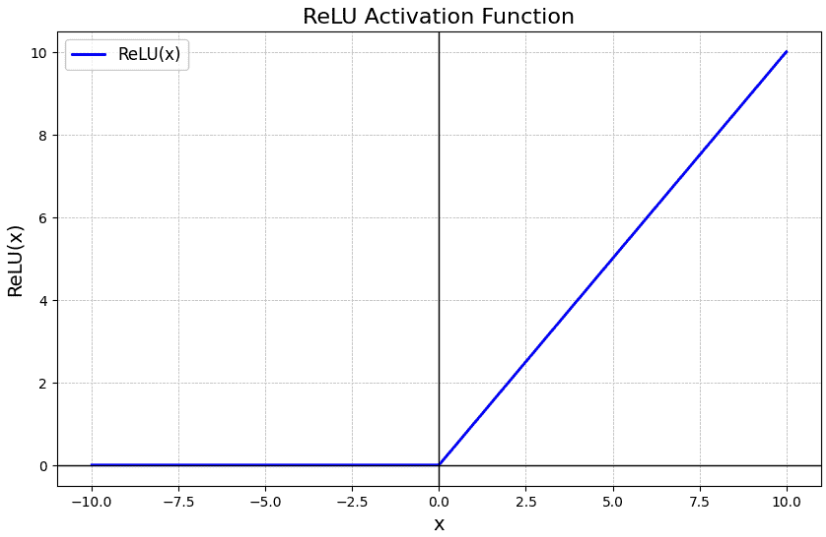


Figure 9 - ReLU Activation Function [11]

ReLU brings computational simplicity in that its simple thresholding operation avoids expensive exponential calculations such as in older functions like sigmoid or tanh. As the gradient is always 1 for positive values, ReLU prevents gradients from shrinking to near-zero values during backpropagation, alleviating the Vanishing Gradient problem.

However, large negative gradients can push a neuron into a permanent state where it always outputs zero and stops learning. This is known as the Dying ReLU problem. Another setback is

that the positive side can grow indefinitely, which sometimes leads to large or exploding activations.

We decided to use the ReLU activation function because it was used in the original U-Net implementation by Ronneberger et. al. and achieved the best results for their task.

3.3 Residual Connections

Residual connections form the basis of the ResNet architecture, introduced by He et. al. in 2015 [10]. Instead of learning a direct input-output mapping, each block in the network learns a residual mapping, which is the difference between the desired output and the input.

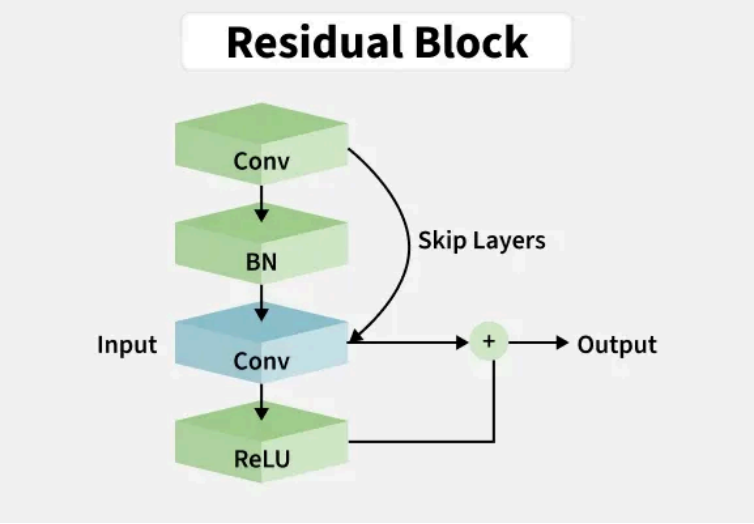


Figure 10 - Residual Connection Diagram [12]

Each residual block in our network consists of a 3×3 convolutional layer with batch normalisation and a ReLU activation. A parallel **shortcut connection** carries the input features to the end of the block, where they are added elementwise to the output of the convolutional layers before a final ReLU activation. When the input and output channel dimensions differ, a 1×1 convolution followed by batch normalisation is applied to the shortcut to ensure dimensional compatibility.

Residual connections provide an additional pathway for gradients during backpropagation, which can reduce optimisation difficulties in deeper networks. They also allow previously extracted features to contribute directly to later convolutions, potentially improving feature preservation.

In our architecture, residual connections are incorporated into four of the encoder downsampling stages, including the bottleneck, and the first three decoder upsampling stages. The initial convolution block and final decoder stage remain standard double-convolution blocks due to **computational overhead** from our large input resolution (1024x2048). These residual

connections complement the original U-Net skip connections, which concatenate encoder and decoder features to recover spatial detail during upsampling. Our specific implementation is discussed further in section 6.2.

3.4 Hyperparameters

The final submission model (v2.0.6) had the following hyperparameters:

Hyperparameter	Value	Description
Crop Size	(512, 1024)	Size of random crops in training batch
Crop Scale	(0.5, 2.0)	Zoom factor range for random crops
Flip	True	Randomly horizontal flip images in training batch
Training Batch Size	8	Number of samples per training batch
Base	32	Base number of channels in the network
CE coefficient	1.0	Scaling factor for Cross-Entropy loss
Lovász coefficient	0.5	Scaling factor for Lovász-Softmax loss
Weight Exponent	0.5	Scaling for Cross-Entropy loss class weights
Max Learning Rate	1e-4	Initial optimization learning rate for scheduler
Min Learning Rate	1e-6	Final optimization learning rate for scheduler
Weight Decay	1e-4	Optimization weight regularization
Epochs	160	Number of training cycles

Table 1 - Table of hyperparameters used

4. Training and Validation

4.1 Loss function

Our loss function combined **Weighted Cross-Entropy** loss and **Lovász-Softmax** loss. The combined loss function aimed to balance pixel accuracy and region segmentation quality.

$$loss = CE + 0.5(Lovasz)$$

4.1.1 Weighted Cross-Entropy

Cross-Entropy loss measures the difference between the prediction and the ground truth per pixel. It penalises the model for assigning low probabilities to the correct class, which encourages accurate pixel classification. This was chosen for our baseline as it is a common starting point for image segmentation.

The dataset had major class pixel imbalance, so without adjustment, the model overpredicted common classes, and underpredicted rare classes. This was addressed by adding class weights to the Cross-Entropy loss. Weights were based on the percentage pixel frequency of classes in the training set, adjusted using **median frequency balancing** and scaled as follows:

$$weight = \sqrt{\left(\frac{median\ frequency}{class\ frequency}\right)}$$

4.1.2 Lovász-Softmax

Although Cross-Entropy is effective for per pixel classification accuracy, it does not directly optimize for the quality of segmentation regions. **Lovász-Softmax** loss [6] sorts pixel prediction errors from largest to smallest, then weights and sums them using IoU based gradients so that the model focuses on errors that most affect segmentation quality.

Adding Lovász-Softmax loss introduced extra computational and memory overhead during training, but this cost was an acceptable tradeoff given the model's performance increase.

Dice loss was considered as an alternative to Lovász-Softmax because it improves overlap between predicted and actual segmentation regions. However, Lovász-Softmax was chosen because it directly targets IoU optimization, which was one of our key evaluation metrics.

4.2 Optimizer

AdamW was the optimizer used for training, with **Cosine Annealing** as the learning rate scheduler.

4.2.1 Adaptive Moment Estimation with Weight Decay

Adam is a computationally efficient optimizer algorithm with little memory requirements [7]. It adjusts model parameters during training using the first moment (mean) and second moment (variance) of the gradients. **AdamW** applies weight decay separately, substantially improving Adam's generalization performance [8].

Alternative optimizers such as SGD with Momentum usually require more tuning and take longer to converge. Given the high computational cost of training runs, AdamW was selected.

4.2.2 Learning Rate Scheduler

Cosine Annealing was used to gradually decrease the learning rate during training. Reducing the learning rate during training helps the model parameters converge towards optimal values without overshooting them. Cosine Annealing was chosen because it was simple and had a smooth transition following the cosine curve. An alternative to try could be linear decay.

4.3 Metrics

Model performance was evaluated using four metrics: **validation loss, pixel accuracy, mean class accuracy,** and **mean Intersection over Union** (mIoU). These metrics were chosen based on the official Cityscapes benchmarks [1] to measure the model's ability to segment images. Validation loss was calculated using the loss function to monitor prediction error. The remaining accuracy-based metrics were calculated from a confusion matrix excluding pixels from ignored classes to evaluate the overall quality of segmentation.

4.3.1 Loss

Both **validation loss** and **training loss** were calculated using the same loss function on different data splits. When compared together, they were used to monitor the model's training progress and to check for overfitting.

The loss in our testing typically ranged from 0 to 3. Decreasing validation loss indicates the model is improving its predictions on unseen data. If the training loss decreases but validation loss remains the same or increases, this is typically a sign of overfitting, as observed from our baseline.

4.3.2 Pixel Accuracy

Pixel accuracy measures the proportion of evaluated pixels that were classified correctly. It is calculated by dividing the total number of correctly classified pixels by the total number of evaluated pixels.

$$pixel\ accuracy\ = \frac{1}{N} \sum_{c=1}^{k} TP_c$$

TP_C = class True Positive, k = total classes, N = total pixels (non-ignored)

This metric provides an overall measure of accuracy, but is heavily influenced by the major class imbalance in the dataset. As roads and buildings dominate the scenes, the model might have high pixel accuracy from predicting these classes even though rare classes are completely lost. This measure ranges from 0 to 1, where 1 is perfect accuracy.

4.3.3 Mean Class Accuracy

Mean class accuracy measures the average of per class accuracies across classes (k). The accuracy for each class (c) is equivalent to per class **recall** and was calculated as follows:

$$class\ accuracy\ =\ \frac{TP_c}{TP_c + FN_c}$$

TP_C = class True Positive, FN_C = class False Negative, c = class

$$mean\ class\ accuracy\ =\ \frac{1}{k}\sum_{c=1}^{k} accuracy_c$$

k = total classes, c = class

This measure gives equal weights to all classes, reducing the influence of dominating classes such as roads or buildings. The measure ranges from 0 to 1, where 1 is perfect accuracy. False positives do not appear in the formula, so a class is not penalised directly for being overpredicted. Mean IoU addressed this limitation.

4.3.4 Mean Class Intersection Over Union

Mean class Intersection over Union (mIoU) is our primary metric used to assess segmentation performance. Intersection over Union (IoU) per class measures the area of overlap between the predicted and true regions, divided by the area of their union. The per class IoU scores were also saved to track per class performance.

$$class\ IoU\ =\ \frac{TP_c}{TP_c + FP_c + FN_c}$$

TP_C = class True Positive, FP_C = class False Positive, FN_C = class False Negative, c = class

$$mean\ class\ IoU\ =\ \frac{1}{k}\sum_{c=1}^{k} IoU_c$$

k = total classes, c = class

This measure ranges from 0 to 1, where 1 is perfect. The mIoU metric is considered most important because it accounts for both false positives and false negatives, providing a more comprehensive segmentation evaluation. It is averaged across classes to account for class imbalance.

4.4 Training Epochs

An epoch in our training consisted of a forward and backward pass over the train set, and a single forward pass over the validation set. The training set was augmented with random crops and flips, while the validation set remained full resolution. The epoch, training loss, validation loss, pixel accuracy, mean class accuracy, and mIoU were saved as history in CSV format.

Automatic Mixed Precision was used to speed up training and reduce GPU memory usage. This allows the use of different numerical precision during computation. Matrix multiplications and convolutions may use floating point 16 bit precision (FP16), while numerically sensitive operations may use floating point 32 bit precision (FP32). Weights remain in FP32 format. A gradient scaler was used to help prevent numerical underflow during mixed-precision training.

Modern training methods use brain floating point (BF16), which uses 16 bits, but has a wider numerical range than FP16. This was not used due to newer hardware requirements.

Every epoch saved a **checkpoint** consisting of the epoch, model, optimizer, scaler, scheduler, best validation loss, best mIoU loss and history. This was done to allow training to resume if interrupted, or to change parameters during different training stages.

A sample output mask of a selected validation image was also produced after every epoch to inspect how the model was doing during various stages of training. At the end, the images were combined into an animated GIF showcasing training progress.

The final **saved model weights** were updated for epochs where the **validation mIoU** score surpassed those of previous epochs. Validation loss was not used for saving weights because mIoU is a more representative metric for segmentation performance.

4.5 Cross Fold Validation

The model is validated on a single split with 3 cities; Frankfurt, Lindau, and Munster. The score depends on which 3 cities happened to be chosen for validation, so if these cities are unusually simple or complex the score may be too high or too low, which is down to luck. The 18 training and validation cities are split by city into 5 folds and 5 separate models are trained. The test cities are excluded. One fold is chosen for validation and the other 4 for training, and this is shuffled and repeated until every city has been used for validation once.The mean is the expected performance on the unseen cities and the standard deviation is how much the folds differ. Ideally all 5 folds should produce roughly the same results, but never exactly the same because each fold is validated on different cities, and each city should have a distinguishable difference. For example, some have busier, more complex scenes and the folds have slightly different sizes. Will the five cities end up having mostly the same performance?

4.5.1 Data Split

We started with 15 training cities and 3 validation cities and combined it into 1 pool of 18 cities and 2,939 total images. The 3 test cities were not included, so they remain unseen data for the final evaluation. It was split by city, not individual images, so that every image from a particular city goes into the same fold. Cityscape images are frames from driving videos, so images from the same city often show the same street. If they were split randomly, near-identical images could end up in both training and validation and the model would effectively have seen the answer, leading to false high scores. This follows the same reasoning we used in the original training, outlined in section 2.3.

The fact that the whole cities are kept together means that rare classes aren't spread evenly. For example, the fifth fold contains only 4 images of trains while the first fold contains 46. This can affect per-class scores of individual folds.

Fold	Validation Cities	Number of Validation Images	Number of Train Images
1	Darmstadt, Strasbourg, Zurich	572	2367
2	Hamburg, Hanover, Ulm, Lindau	598	2341
3	Cologne, Weimar, Frankfurt	563	2376
4	Aachen, Bremen, Erfurt	599	2340
5	Bochum, Krefeld, Monchengladbach, Tubingen, Munster	607	2332

Table 2 - Cities and number of images in each fold

4.5.2 Training Setup

The same U-Net architecture, loss, optimiser and learning-rate schedule were used during crossfold validation so that the only thing that changes between folds is the data. The differences only come from the cities, not the training settings. Every fold starts from a brand-new model, optimizer, and schedule, with nothing carried over from the previous fold to prevent leakage. Otherwise a later fold’s model would already have trained on that fold’s validation cities. The random seed was the same for every fold so all the models start from the same initial weights, ensuring that the differences come from the data and not by random luck at all.

As you can see from Table 3, five full training runs like the main model would take over 32.5 hours, which is not efficient. To shorten training time, images were downsized and only 40 epochs were used per fold instead of 160. The only data augmentation allowed during training were flips. Random crops were disabled to reduce the training time to a manageable duration

while still allowing sufficient learning. The batch size is 32 so that smaller images fit more per batch on the GPU, which also speeds up training.

The model is evaluated on the folds' validation cities after every epoch using the same rules we used for the main model, so the epoch with the highest validation mIoU is kept. Due to the lower resolution, fewer epochs, and larger batch size, the scores aren't directly comparable to the main model.

Setting	**Main model**	**Cross-Validation**
Training input	512x1024	256x512
Validation input	1024x2048	256x512
Base	32	32
Batch size	8	30
Epochs	160	40 per fold
Augmentation	Crops + Flips	Flips
Loss / optimizer	Weighted CE + 0.5 Lovasz / AdamW / cosine 1e-4->1e-6	Weighted CE + 0.5 Lovasz / AdamW / cosine 1e-4->1e-6
GPU / time	Rtx 4080 Super / ~ 6.5h	RTX 4080 Super / ~ 20 min per fold

Table 3 - Settings

4.5.3 Discussion

To answer my initial question, yes, the five folds give similar results. The model performed consistently on different groups of cities so we can confirm that it generalises rather than only overfitting to certain cities. This suggests our original validation split gives a fair estimate because a different choice of cities would likely have given a similar score.

In the early epochs, the model learns the large common classes, such as road, building, and vegetation, which look similar in every city so they all improve equally. In later epochs, it starts learning smaller, harder classes and details and that's where the differences between cities start to matter.

Mean class accuracy spreads more than pixel accuracy because every class counts equally there including some rare classes that are unevenly spread across folds. The fifth fold has only 4 images containing trains but it is still one of the best folds because rare classes affect individual class scores while barely moving the overall results.

There was no overfitting, which is evident in how the validation loss curve levels off for every fold and doesn't rise again. The training loss being below validation loss is normal and it only becomes a problem if validation loss starts to rise. The horizontal flips and the class weighted loss helped prevent overfitting.

The best were epochs 33-39 and folds 4 and 5 were still improving slightly so more epochs would most likely raise the score further.

These scores are lower than the model because of the reduced resolution and training so they show consistency but not better performance.

4.5.4 Limitations

In conclusion, the results should not be compared with the main model due to the reduced training settings used (lower resolution, fewer epochs, and a larger batch). Crossfold validation shows how consistent the model is, not its best score. Each fold's best epoch was chosen using its own validation data [13].

5. Evaluation of Results

5.1 Cross Fold Validation Results

Fold	Best Epoch	Mean IoU	Pixel Accuracy	Mean Class Accuracy	Validation loss	Training Time (mins)
1	33	0.4280	0.8757	0.5583	1.2085	20.0
2	38	0.4276	0.8772	0.5566	1.2100	19.4
3	34	0.4435	0.8862	0.5741	1.1488	19.4
4	35	**0.4594**	0.9014	0.5920	1.0215	18.9
5	39	0.4572	**0.9018**	**0.5950**	0.9823	19.3
mean		0.4431	0.8885	0.5752	1.1142	
std		0.0152	0.0127	0.0181	0.1064	

Table 4 - Results of each fold's best epoch

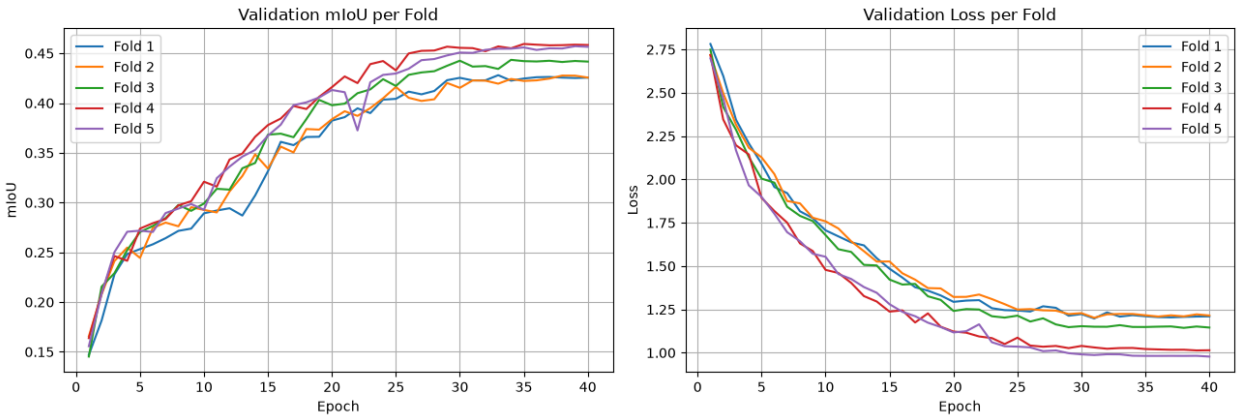


Figure 11 - Cross-validation mIoU and loss curves

Table 4 shows each fold's best epoch and its scores along with the mean and standard deviation across all 5 folds. Figure 11 shows validation mIoU and validation loss for every fold over the 40 epochs. The main result was an mIoU of 44.3% with a standard deviation of 1.5%. The lowest mIoU scores came from the first two folds, the minimum being 42.8%. The fourth fold produced the higher mIoU at 45.9%, giving an mIoU range of about 3 percentage points. Mean pixel accuracy is 88.9% with 1.3% standard deviation, and was stable across all folds. Mean class accuracy was 57.5% with a standard deviation of 1.8%, slightly more spread than mIoU. Validation loss was 1.11 with deviation of 0.11, the lowest being in folds 4 and 5 and highest in folds 1 and 2.

In Figure 11 the curves show that all 5 folds improved at almost the same rate for the first 20 epochs and after that they were separated into 2 groups: folds 4 and 5 end higher in mIoU and lower in loss, folds 1, 2 and 3 end lower in mIoU. The best epochs were late, between 33-39, and folds 4 and 5 were still showing signs of improvement at epoch 40. Validation loss levels off for every fold and never rose in the end.

The folds all give similar results with a spread of about 3 percent, which is a satisfactory result. It shows that our results are not dependent on the choice of split and that our model should **generalise well to unseen data**.

5.2 Training Performance

The model took approximately **6hrs 30mins** to train 160 epochs using the Nvidia RTX 4080 Super GPU with 16 GB of VRAM.

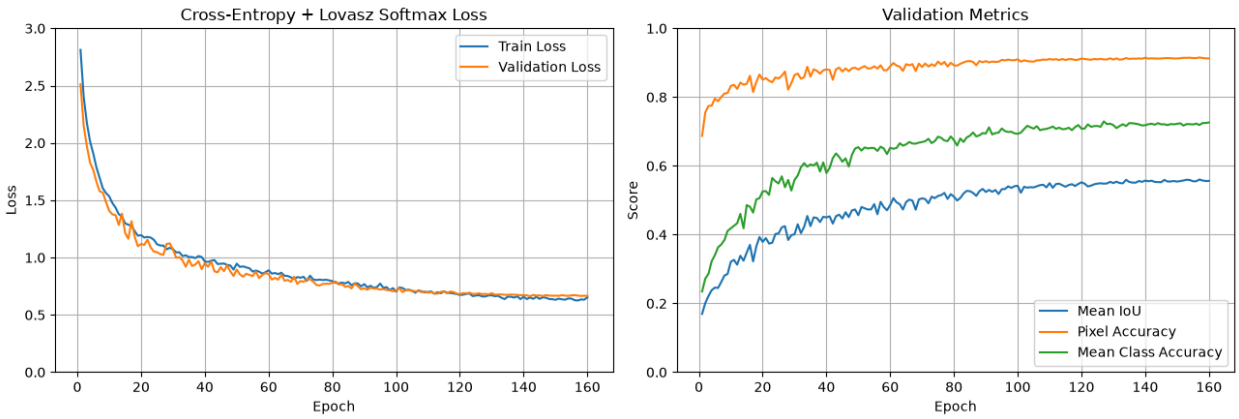


Figure 12 - v2.0.6 Training and validation loss show no signs of overfitting.

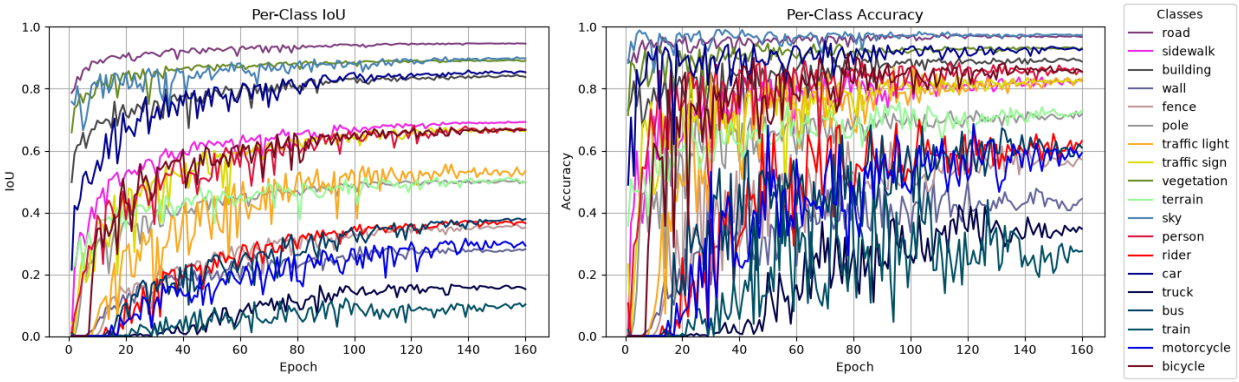


Figure 13 - v2.0.6 All classes appear by epoch 20.

The training and validation loss overlap the whole way, showing good generalisation. There are no signs of overfitting.

Epochs 0-20 - The learning rate starts highest at 1e-4. Loss drops quickly from **~2.6** down to **~1.2**. Class metrics scores appear almost instantly for common classes, while rare classes such as train or truck take longer. After the first 20 epochs all classes have non-zero accuracy. The mean IoU increases from **0.17** to **0.40**, mean class accuracy (recall) from **0.23** to **0.52**, and overall pixel accuracy from **0.68** to **0.85**.

Epochs 20-100 - The model makes slower progress reducing loss down to **~0.7**. The learning rate decreases steadily following the cosine annealing curve. Rare class accuracy (recall) spikes heavily as the decision boundary flips between under- and overpredicting. Possible explanations could be class weights set too high or rare classes being missed by the random crop process in certain epochs. IoU spikes less because under- and overpredicting cancel out, while recall only sees the missed pixels.

Epochs 100-160 - The learning rate is low and loss only goes down to **~0.6**. The performance gains flatten and epochs add very little improvement. Accuracy spikes reduce due to the lower

learning rate. The remaining loss most likely comes from model capacity, reduced resolution or class balancing setup.

The model saved the final weights at epoch **157**, reaching the peak validation mean IoU score of **0.56**. The mean class accuracy (recall) for that epoch was **0.72**, while the overall pixel accuracy was **0.92**. These weights were used to run the test set evaluation.

5.3 Final Test Evaluation

Train ID	Class	IoU (%)	Accuracy (Recall) (%)
0	road	95.5016	97.2153
1	sidewalk	70.4764	82.0344
2	building	85.8947	91.2153
3	wall	33.9570	53.0536
4	fence	36.5359	57.1607
5	pole	44.1294	65.5635
6	traffic light	50.7923	80.4248
7	traffic sign	64.2553	80.1419
8	vegetation	87.5672	91.8035
9	terrain	61.7409	83.0927
10	sky	91.9723	96.9487
11	person	70.0300	81.6907
12	rider	28.4743	66.3065
13	car	86.4109	95.5431
14	truck	19.0423	32.5865
15	bus	23.2668	58.7699
16	train	28.4003	48.7717
17	motorcycle	43.3096	69.2258

18	bicycle	54.7422	75.9947

Table 5 - 2.0.6 Per-class test evaluation results.

v2.0.6 - Test Evaluation:
Pixel Accuracy: **92.0421***%, Mean IoU (mIoU):* **56.6579***%, Mean Class Accuracy:* **74.0812***%*

The aggregate test evaluation results match validation results quite well, with the largest difference being mean class accuracy (recall) at **~2%** higher. Generalisation is solid and the saved weights did not overfit the validation set.

Inspecting the per class results shows **recall** is constantly higher than **IoU**, which is a likely sign that the model **overpredicts rare classes**. Common classes such as road, building or vegetation are balanced, while rare classes such as bus, rider or traffic light have larger differences between the two metrics.

Comparing the weakest validation set class to the test results shows a discrepancy. During validation, the **train** class consistently scored lowest, finishing with **0.10** IoU and **0.27** recall, while the test results show a significantly higher **0.28** IoU and **0.48** recall. The train class appears in least images in the validation set. This difference most likely reflects the small number of samples rather than a genuine model improvement.

The pole class appears commonly but does not occupy many pixels. The model reduces a 1024x2048 resolution image down to 64x128 at the bottleneck. A pole the width of a few pixels could disappear entirely at this depth. The U-Net skip connections aim to mitigate this by passing high resolution encoder features to the decoder. The results suggest that the skip connections work as the pole class scores **0.44** IoU.

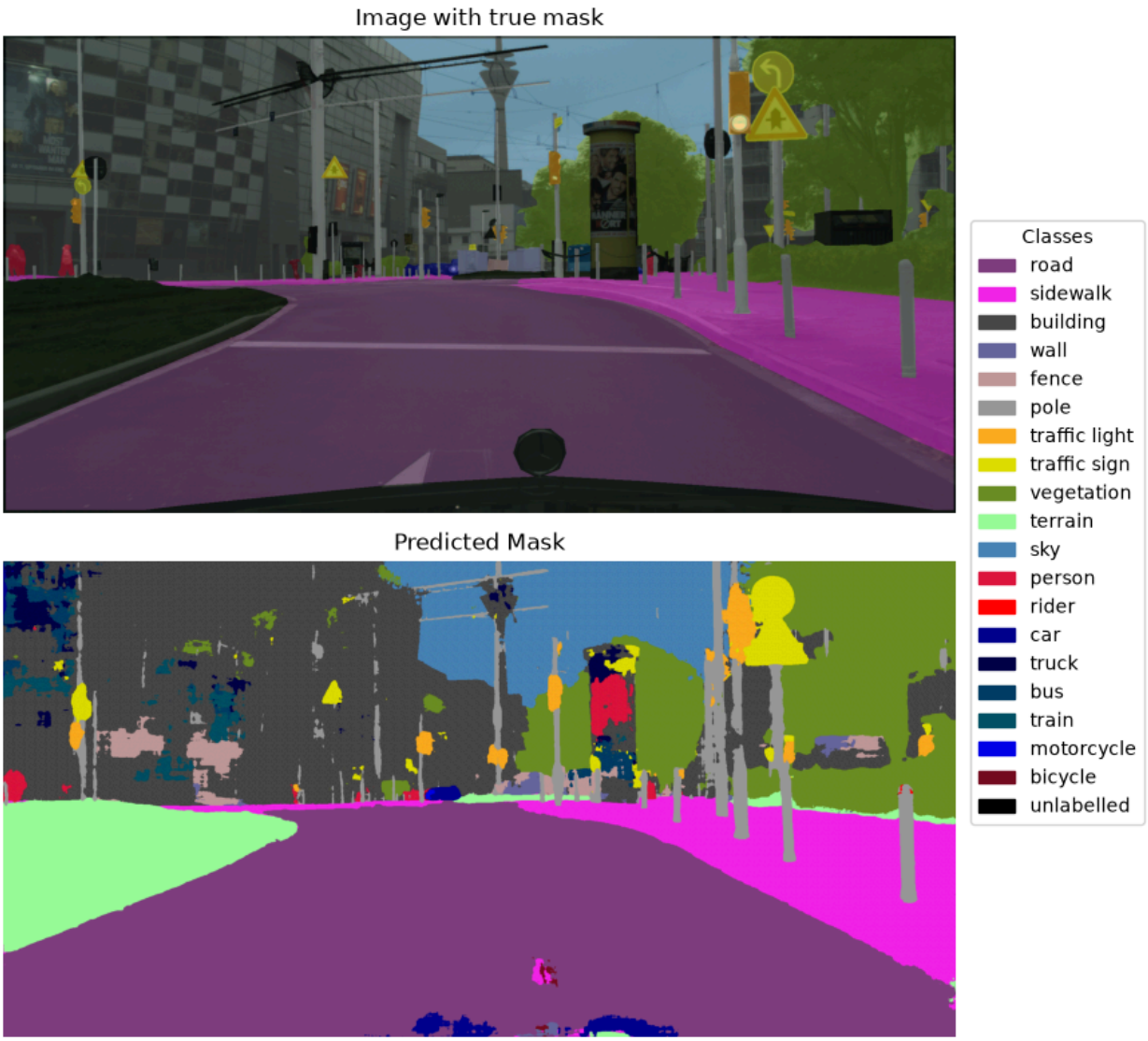


Figure 14 - v2.0.6 - Sample output mask from test set.

Multiple sample output masks were generated to visually inspect segmentation quality. Figure 14 demonstrates a few of our observations.

The uncoloured true mask regions should be ignored in the sample output, as the model was not trained to handle these classes. Evaluation metrics handle this automatically.

Large common classes such as road, sidewalk or vegetation are captured well. These regions are clean, smooth and match their high IoU scores. Objects closer to the camera are detected better than objects further away, which occupy less pixels. This is particularly noticeable with poles and traffic signs. Traffic signs and traffic lights come out as large rounded blobs with fuzzy edges.

The large tiled building on the left side of Figure 14 is speckled with incorrect rare class prediction patches, that have high recall compared to IoU such as bus, train and truck. This supports the hypothesis that the model overpredicts rare classes.

These incorrect rare class patches appear with a checkerboard pattern. This is a likely sign of uncertain or confused predictions. These checkerboard artefacts could also stem from the upsampling transpose convolution, which is known to cause checkerboard patterns. Using bilinear convolution could reduce the occurrence of this pattern.

The model is very good at identifying persons, even when positioned far in the background of an image. In Figure 14, the model captures the human face in the advertisement near the centre. This region is unlabelled, and does not count towards evaluation.

We evaluated our results against A. Paszke et. al's 2016 paper [14] which establishes the **ENet** (Efficient Neural Network), a fully convolutional network created specifically for tasks requiring low latency operation. ENet was trained on three different image segmentation datasets, including Cityscapes, and provides individual results for its performance on Cityscapes images. Table 4 in the paper compares the official Cityscapes test set benchmarks for ENet and SegNet.

Model	**Official Cityscapes Benchmark Class IoU**	**Our Test Set IoU**
SegNet	56.1	-
ENet	**58.3**	-
Our U-Net	-	56.6579

Table 6 - Mean Class IoU of SegNet, ENet, and our U-Net on the Cityscapes dataset

As the Cityscapes official test set used for benchmarking does not contain annotated masks, we cannot provide the benchmark-comparable results from our model without uploading it to the website. Instead, we use our final mIoU score from our custom unseen test set which consists of three cities from the official training and validation sets. Evident in Table 6, our U-Net implementation scored **~0.56% higher on mIoU than SegNet**, and **~1.64% lower than ENet**.

6. Independent Experimental Work

6.1 Training Experimentation

Due to high resolution requirements for capturing detail, experimentation runs were limited by resources. This limitation reduced our hyperparameter tuning potential. Test set evaluation was performed for comparison. Experimentation was driven by validation results, not test results.

The project began with a basic U-Net implementation as the baseline [4]. The model was trained on the full resolution training set every epoch, with no augmentation. This resulted in a small batch size, slow training time and overfitting. The baseline took 5hrs to complete 20 epochs.

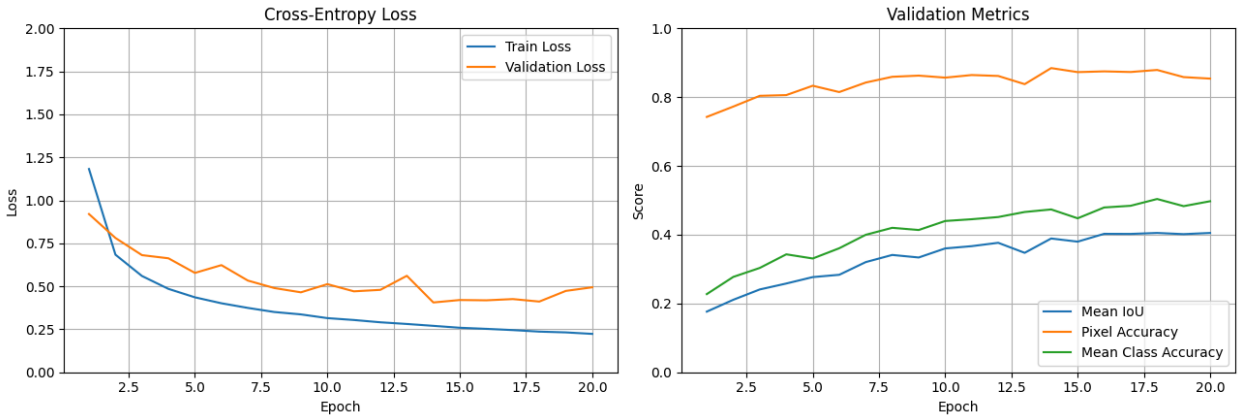


Figure 15 - v1.0.2 baseline trained on full resolution, shows signs of overfitting.

v1.0.2 baseline - Test Evaluation:
Pixel Accuracy: 90.0533%, Mean IoU (mIoU): 39.9852%, Mean Class Accuracy: 48.5771%

The first major improvement in training performance came from using smaller resolution crops and horizontal flips. Any transformation done to an image was also done to the matching mask. Training set images were cropped and reduced to **512x1024** instead of the original **1024x2048**. Each image had **25%** of the original image's pixels. Taking up less memory, the training batch size could be increased from **1** to **6**. The validation and test set images and batch sizes were not altered.

These changes had multiple effects on training. Each epoch took less time due to the increased batch size. This allowed the model to train more epochs in the same amount of time, which meant more weight updates. Since the training set now contained random crops, each epoch consists of a unique set of images, which in our case eliminated most overfitting. Potential downsides included reduced general spatial awareness from zoomed-in crops and rare classes being skipped in certain epochs. This new approach achieved similar validation results in just **~60%** of the baseline's training time.

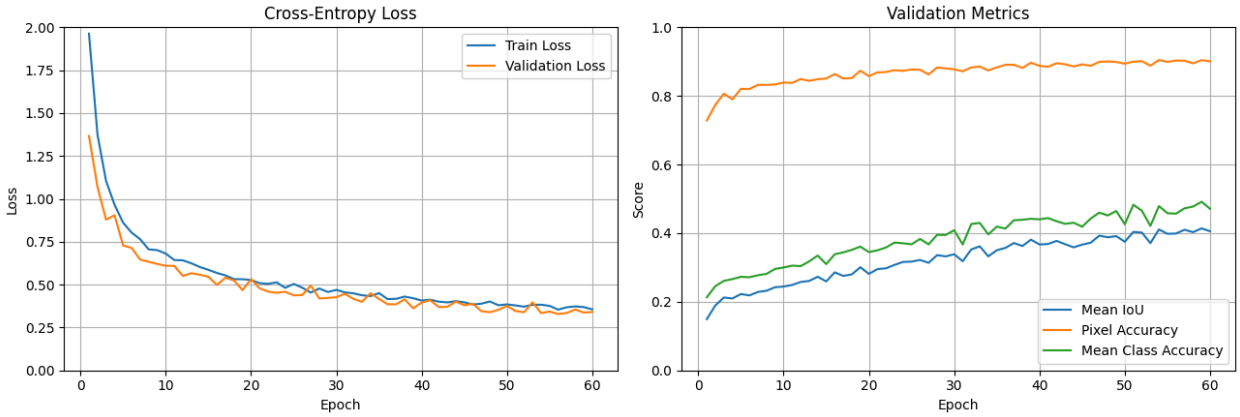


Figure 16 - v1.1.0 trained on smaller crops with random flips, shows no signs of overfitting.

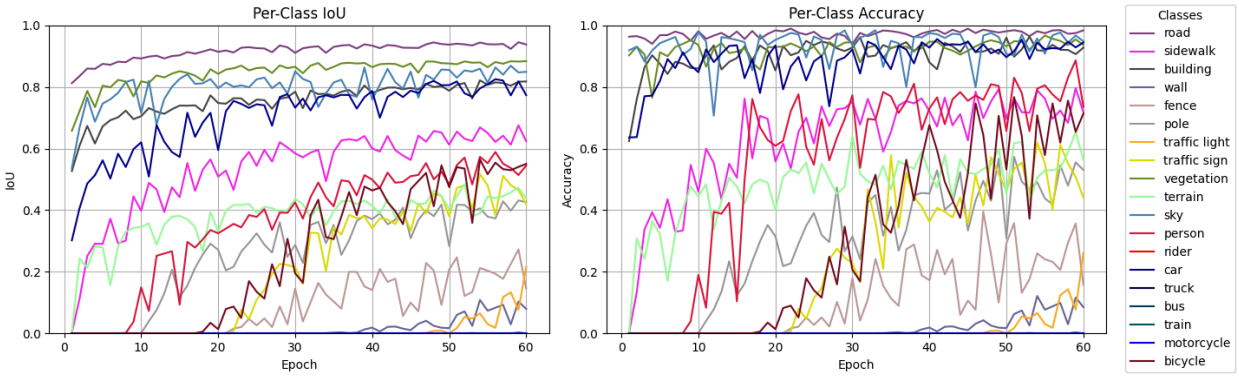


Figure 17 - v1.1.0 shows certain rare classes appearing in later stages of training, while others are missing entirely. The spikes seen in accuracy are likely due to random crops omitting rare classes during particular training epochs.

v1.1.0 - Test Evaluation:
Pixel Accuracy: 90.9222%, Mean IoU (mIoU): 39.6777%, Mean Class Accuracy: 45.1417%

Further minor improvements arose from combining the separate validation loss and evaluation metrics forward passes into a single forward pass, saving progress with checkpoints, using automatic mixed precision and saving a sample output every epoch.

Weights were added to the Cross-Entropy loss at first using only unscaled median frequency balancing. For example, the motorcycle class weight was **390x** larger than the road class. As a result the model overpredicted rare classes and did not achieve higher validation results.

The median frequency-balancing weights were adjusted to be scaled by taking the square root, which resulted in increased pixel accuracy by **+0.83%**, mean class accuracy by **+13.59%** and mean IoU by **+6.3%** measured at 60 epochs. This was a significant mean class accuracy increase. The model discovered rare classes much earlier in training.

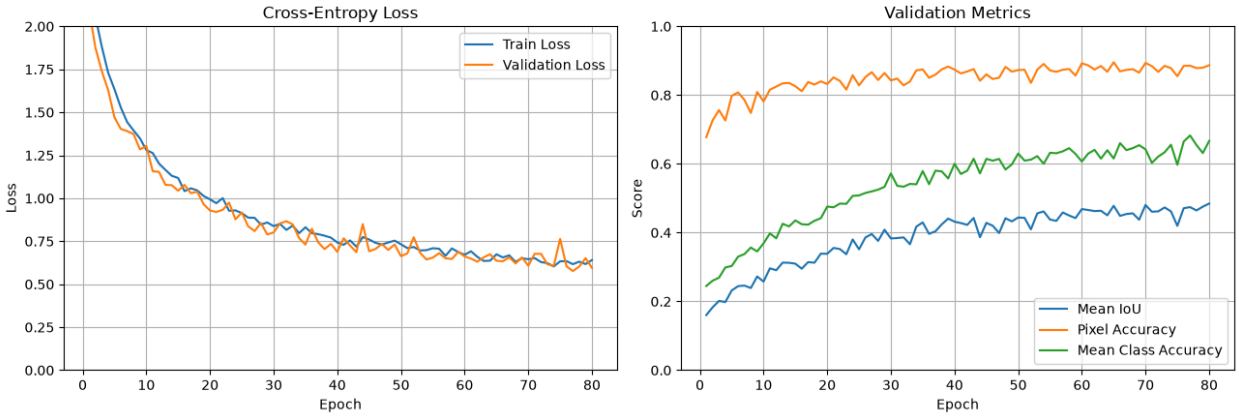


Figure 18 - v1.2.2 shows improvement in mean IoU and mean class accuracy at 60 epochs.

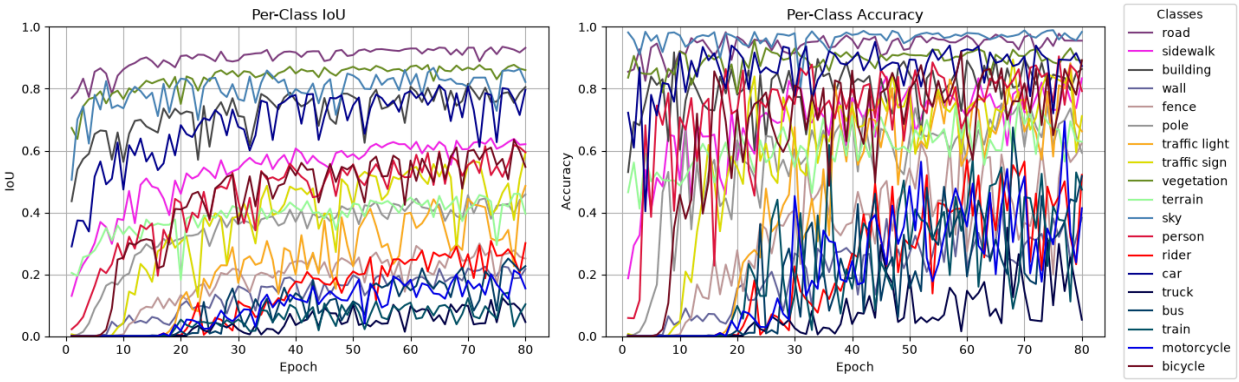


Figure 19 - v1.2.2 shows all classes captured by epoch 20.

v1.2.2 - Test Evaluation:
Pixel Accuracy: 89.5973%, Mean IoU (mIoU): 49.1373%, Mean Class Accuracy: 68.8550%

The addition of Lovász-Softmax to the loss function increased the validation pixel accuracy by a further **+1.36%**, mean class accuracy by **+1.52%** and mean IoU by **+3.64%**, measured at 80 epochs. Cosine Annealing learning rate adjustment was introduced to help the model stabilize towards the end of training. Running the training for longer, i.e. for more epochs, resulted in higher accuracy scores.

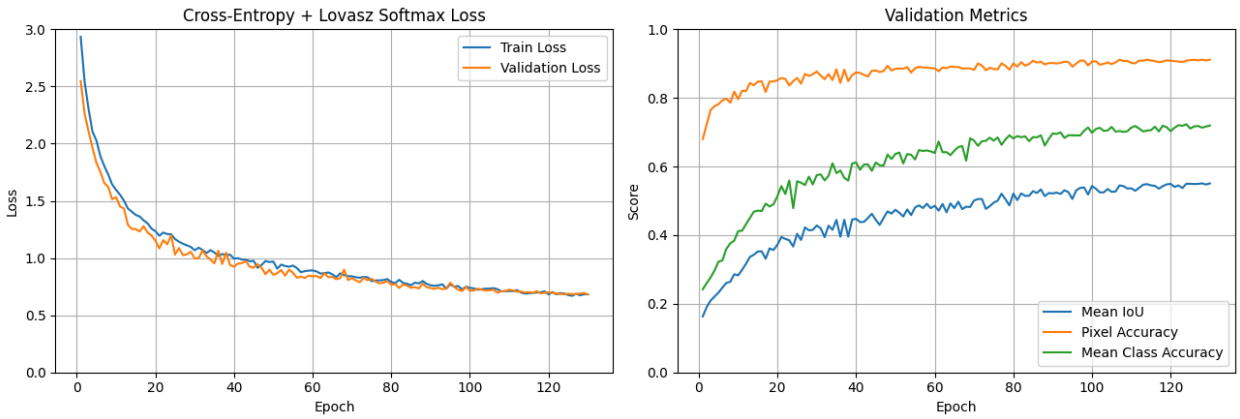


Figure 20 - v1.3.0 shows minor accuracy improvements after 100 epochs, decreased learning rate.

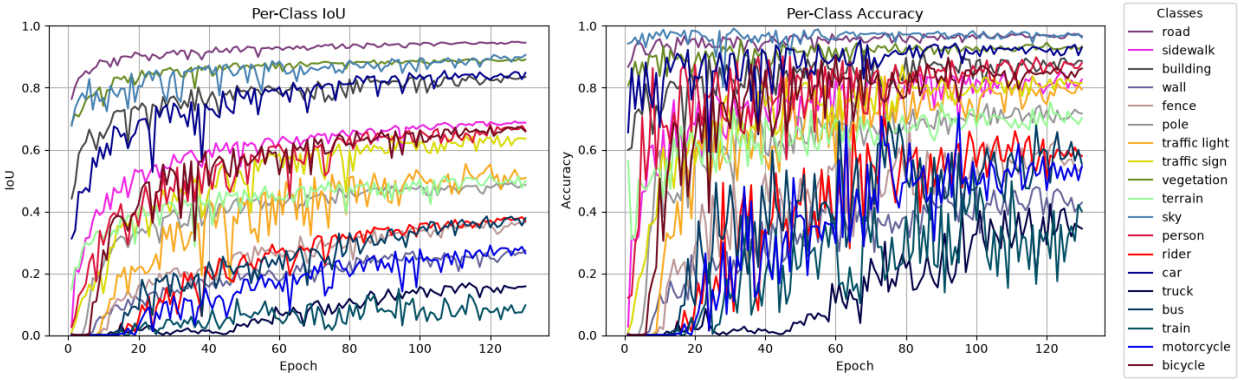


Figure 21 - v1.3.0 shows mIoU spikes decreasing with the learning rate.

v1.3.0 - Test Evaluation:
Pixel Accuracy: 91.7056%, Mean IoU (mIoU): 55.6871%, Mean Class Accuracy: 73.7495%

6.2 Model Architecture Experimentation

6.2.1 Baseline

In order to experiment with the original model architecture on limited GPU resources, we created a downscaled version of the full model training similar to that used for cross-validation. This way, we could test different additions and make changes without having to wait 4-6 hours to see the final results. Upon setting the downscaled test conditions, we ran the model training with no adjustments to generate a **baseline**.

Setting	**Main Model Training**	**Model Experimentation**
Training input	512x1024	256x512

Validation input	1024x2048	512x1024
Base	32	32
Batch size	8	6
Epochs	160	20
Augmentation	Random Crops & Flips	Random Crops & Flips
Loss / optimizer	Weighted CE + 0.5 Lovasz / AdamW / Cosine 1e-4->1e-6	Weighted CE + 0.5 Lovasz / AdamW / Cosine 1e-4->1e-6

Table 7 - Hyperparameters for model experimentation

Under these conditions, we were able to train 20 epochs on an NVIDIA GeForce RTX 2050 GPU in about 65 minutes. The baseline test results were as follows:

Baseline Test Evaluation:
Pixel Accuracy: 84.2711%, Mean IoU (mIoU): 29.6125%, Mean Class Accuracy: 38.8709%

Rare classes produced 0% mean IoU and class accuracy on this test set, as 20 epochs was too short a training duration for the model to learn deeply.

6.2.2 Multi-Scale Dilated Bottleneck

The first experiment we ran on the U-Net architecture was adding layers to its **bottleneck**, the deepest part of the network. As the bottleneck sits after four to five blocks of downsampling, its input feature maps are the smallest, making convolution operations the least computationally expensive here. We attempted to take advantage of this by implementing a **multi-scale dilated bottleneck**. A 3x3 dilated convolution spaces out the sampled kernel positions to sample adjacent pixels while maintaining its nine spatial weights. This allows the model to extract features at different spatial scales while computing convolutions on only a fraction of the original resolution.

We created a multi-scale dilated bottleneck with two paths; one with dilation set to 1 (each kernel pixel is spread one pixel outwards) to learn local features, the other with a dilation of 2 (each kernel pixel is spread two pixels outwards) to learn features from a wider spatial neighbourhood. The path converged on a 1x1 fuse convolution to combine the information from both branches and restore the original channel count.

We then trained the model for 20 epochs under the same conditions as the baseline and achieved the following results:

Multi-Scale Bottleneck Test Evaluation:
Pixel Accuracy: 84.2852%, Mean IoU (mIoU): 30.1880%, Mean Class Accuracy: 39.1430%

The multi-scale dilated bottleneck took 13 minutes longer to train (78 minutes in total) but achieved only a very minor improvement in mean IoU and mean class accuracy. The rare classes showed no improved learning with this addition, remaining at 0% mean IoU and class accuracy. It may have been the case that the dilated convolutions at the bottleneck only provided a modest amount of additional useful information compared to the downsampling and skip connections. The added parameters in the bottleneck may also have needed more than 20 epochs to train effectively.

6.2.3 Squeeze-and-Excitation Blocks

A **Squeeze-and-Excitation (SE) block** is a **channel-attention mechanism** that learns a weight for each feature channel in a convolution and uses it to rescale a feature map. This ensures that channels that are deemed more important or influential to the model's learning are prioritised when passed forward through the layers.

The squeezing step applies global average pooling across the spatial dimensions to compress each channel into one number. This creates a global summary descriptor for each channel. The excitation step passes this channel vector through a bottleneck made up of two layers, which learns channel importance through cheap 1x1 convolutions. The output is a set of per-channel attention weights ranging from 0 to 1. Each channel is then multiplied by its corresponding weight, hereby **amplifying informative channels** and suppressing noisy or unimportant ones.

We added SE blocks to the model architecture, first on only the baseline without the multi-scale dilated bottleneck with a training time of 73 minutes:

Squeeze-and-Excitation Block Test Evaluation:
Pixel Accuracy: 70.5821%, Mean IoU (mIoU): 23.2715%, Mean Class Accuracy: 34.2391%

Then, right after the bottleneck with a training time of 68 minutes:

Multi-Scale Bottleneck with SE Block Test Evaluation:
Pixel Accuracy: 73.8601%, Mean IoU (mIoU): 24.7463%, Mean Class Accuracy: 35.7029%

Both tests achieved weaker results across all metrics than the baseline, again with 0% mIoU and class accuracy for the rare classes. Potential reasons for this include global average pooling diluting rare-object signals, the fact that SE does not directly model spatial importance, and channel suppression possibly interfering with feature learning. Additional complexity is not automatically beneficial.

6.2.4 Residual Connections

As introduced in section 3.3, we experimented with **residual blocks** in our U-Net's deeper layers. The goal was to see if this would improve optimization by creating a more direct gradient path (shortcut) through **shortcut connections** during backpropagation.

If the number of input channels in a convolution matched the number of output channels, we set the shortcut to the layer's input. Otherwise, we would perform a 1x1 convolution to map the input to the required shape, followed by batch normalisation, and that becomes the shortcut. The final ReLU activation is carried out over the concatenation of the original convolution and this shortcut, creating the residual connection.

These were our results after implementing residual connections in the baseline and running the training with 20 epochs, which took 75 minutes:

Residual Block Test Evaluation:
Pixel Accuracy: 84.8450%, Mean IoU (mIoU): 32.4676%, Mean Class Accuracy: 41.8035%

We then tested residual connections with the multi-scale dilated bottleneck, for a training duration of 74 minutes:

Residual Block with Multi-Scale Bottleneck Test Evaluation:
Pixel Accuracy: 84.6585%, Mean IoU (mIoU): 31.9378%, Mean Class Accuracy: 41.4921%

Both tests show improvements over the baseline, especially the residual blocks without the multi-scale bottleneck. Finally, the rare classes rose from 0% mIoU and class accuracy to 0.02% by the end of the 20 epochs. Unlike multi-scale convolutions and SE attention, residual connections modify how features and gradients propagate, rather than introducing a new feature-selection mechanism. This can make optimisation easier and convergence occur sooner, without relying on the network learning attention weights or exploiting extra spatial context.

Based on these results, we decided to keep residual connections in our model structure and run a full training on the default settings (full resolution input images, 160 epochs).

130-Epoch Test Evaluation (without Residual Connections) - 4 hours training time:
Pixel Accuracy: 91.7056%, Mean IoU (mIoU): 55.6871%, Mean Class Accuracy: 73.7495%

160-Epoch Test Evaluation (with Residual Connections) - 6.5 hours training time:
Pixel Accuracy: 92.0421%, Mean IoU (mIoU): 56.6579%, Mean Class Accuracy: 74.0812%

We found that this addition increased pixel accuracy by about 0.34%, mean IoU by 0.97%, and mean class accuracy by 0.33%. This is a **miniscule improvement** despite an additional 30 epochs, far smaller than predicted given the positive results produced during the downscaled experimentation. Residual connections serve to improve optimisation and training in deep neural networks with over 50, even 150 layers. Our model is only 30 layers deep, which we concluded was **not deep enough** for this concept to bring a notable improvement.

7. Estimated Level of Difficulty

We found this assignment particularly challenging due to our choice of task, model, and dataset. As U-Nets were not covered in the module lectures, we had to study the architecture independently. Our decision to train a U-Net from scratch rather than using transfer learning added to the difficulty. Semantic segmentation is computationally demanding, as the model must classify every pixel in an image. The Cityscapes dataset contains images with a resolution of 1024 × 2048, approximately four times the pixel count of the images used in the original U-Net implementation. Consequently, we spent considerable time experimenting with the model architecture, training configuration, and hyperparameters to achieve satisfactory performance within our limited GPU resources. Despite these challenges, we found the project highly insightful and valuable to our future careers.

8. Statement of Work

Igor split the data into train/validation/test sets, prepared dataloaders, implemented the baseline U-Net model, researched and created evaluation metrics, optimized training, added automatic mixed precision, introduced random crops and flips, added training checkpoints and history, graphed validation metrics, experimented with loss functions and optimizers, introduced variable learning rate, experimented with the normalisation and upscaling layers, created the network structure diagram, evaluated final results.

Emily carried out the exploratory data analysis, identified the severe class imbalance and lack of duplication and leakage across cities in the dataset to allow us to split our train, validation, and test sets by city, visualised important observations in the data, calculated pixel frequencies which were later used for weighted cross-entropy, experimented with model architecture: tested a multi-scale dilated bottleneck, squeeze-and-excitation blocks, and then residual connections which were kept in the final model.

Nikolay designed the 5-fold cross-validation,wrote notebook 4 -crossfold validation, splitting dataset into balanced folds divided by city to avoid data leakage, built the per-fold train folders, produced results tables, ran crossfold validation.

9. Use of Generative AI

Generative AI was used in this project for coding, troubleshooting and explanation. Many implementations remain based on sourced material or documentation.

Prompt: Semantic segmentation U-Net project based on the 19 classes from the Cityscapes dataset. What are the most suitable metrics to evaluate performance? Explain each method and how to implement it. Take into consideration the official benchmarks from the cityscapes website.

Response: The response suggested Pixel Accuracy, Mean IoU (mIoU) and Mean Class Accuracy and explained each. Provided sample implementations for each.
Usage: These metrics were checked and understood. The code was used as a base implementation for metrics used to evaluate model performance.

Prompt: How to save PyTorch model training progress and restore it. How to save and display training history including validation and evaluation metrics.
Response: Explained checkpoints and provided as sample implementation. Provided code to restore from checkpoints. Provided an example of how to save training metric history.
Usage: Code was easy to understand and used directly in the project.

Prompt: U-Net model on the cityscapes dataset. How to save a sample output .png during every training epoch and then generate a gif of training progress.
Response: Simple implementation with two functions.
Usage: Code was easy to understand and used in the project. Slightly modified to apply the official cityscapes colour scheme to the output.

Prompt: Review this training loop for correctness and suggest improvements.
Response: Suggested the use of AMP (Automatic Mixed Precision) for training on a GPU and explained the importance of using a scaler to help prevent numerical underflow. Provided a sample AMP implementation.
Usage: Applied the suggested changes adding comments to understand.

Prompt: How to implement random crops and horizontal flips based on this approach.
Response: Provided a sample implementation of both.
Usage: Code was modified and implemented. As a result data transforms were moved into the dataset class and new parameters were made such as flips and crop scale. The batch size for training and evaluation was split into different variables since evaluation was on full resolution images but training was on smaller crops, allowing larger batches.

Prompt: How to save per class metrics from evaluation into the history csv and display as separate graphs.
Response: Simple implementation.
Usage: Used directly and commented.

Prompt: How to implement class frequency weights to deal with class imbalance.
Response: Suggested the use of median frequency balancing and provided a simple implementation using the frequencies from the exploratory data analysis. Explained how to adjust weights accordingly.
Usage: Frequencies from exploratory data analysis used and adjusted. Added one parameter to the loss function.

Prompt: How to implement scheduled learning rate?

Response: Simple example of how to implement and a comparison between different learning rate schedulers and which are most suitable.
Usage: Decided to take the Cosine Annealing learning rate scheduler after further research.

Prompt: Dice loss vs Lovasz softmax loss, explain both and provide sample implementations.
Response: Explained the difference between the two. Provided sample implementations.
Usage: Decided to use Lovasz as it directly focuses on IoU, which is the most important metric. Used implementation with small adjustments.

Prompt: How to do a full exploratory data analysis on an image dataset with masks and colours for image segmentation? It has 19 classes and the dataset consists of images for multiple different cities where the city name is in the image name.
Response: Suggested key topics to look at and provided code implementations for various graphs and figures to help visualise the dataset.
Usage: Implemented the suggested code with small adjustments to account for Cityscapes-specific labels and colours.

Prompt: How do I interpret and explain each of these [EDA] results and what to do to the data depending on each result?
Response: Since I didn't provide the results directly, the model gave potential outcomes for each graph with suggested courses of actions to handle each imbalance or issue in the data.
Usage: I compared the listed outcomes with the actual outcome and wrote up the interpretation of each graph in the notebook, along with the suggested solution. From this I was able to identify a severe class imbalance in the Cityscapes dataset and so decided to calculate the percentage pixel frequency for each semantic class across the dataset so that we could later use these as weights in our weighted Cross-Entropy loss function.

Prompt: Explain every line and concept in this U-Net CNN model architecture and then suggest improvements (such as something with the bottleneck, residual modules, attention modules, etc) that would improve performance without greatly increasing complexity and training time.
Response: Explained the original U-Net architecture that we'd been using, how U-Nets work in general, and suggested the following improvements along with code implementations: residual connections, a multi-scale dilated bottleneck, Squeeze-and-Excitation (SE) attention blocks, attention gates on skip connections, and deep supervision.
Usage: I understood and annotated our existing architecture, then one-by-one implemented and tested different combinations of the given suggestions until I found one that gave the best improvement in our evaluation metrics without adding to the training time for the small, fixed sample I had chosen.

Prompt: Instead of writing the cross-validation notebook for me, explain step by step what to change in notebook 3 so I can learn it.
Response: A step-by-step plan: copy the notebook, define the folds, build the fold folders, change the settings, make the data loaders, write the fold loop, then produce the results table

and plot.
Usage: Followed to build notebook 4 on cross validation.

Prompt: Using the city image counts from notebook 1, suggest how to split the 18 training and validation cities into 5 folds of similar size.
Response: A 5-fold split by city, with 563–607 images per fold.
Usage: The fold split was used directly in the cross-validation notebook.

Prompt: Suggest more comments I should add to my code.
Response: Gave a list of comments for all the cells I should add.
Usage: Some of the comments were enhanced with AI and the rest were written by me.

Prompt: Help me plan the cross-fold validation section of the report and check my draft for factual mistakes.
Response: Gave a section structure, notes and data tables (fold split, settings, results), and pointed out factual issues in my raft without rewriting it.
Usage: The tables were used in the report; the text was written by me.

Prompt: My code for the training loop doesn't work. Could you tell me what's wrong with it?
Response: Fixed the optimizer and scheduler per fold.
Usage: Some code used and ran in the notebook.

Prompt: Should the 5 fold results be the same? And if validation loss is higher, is that worse?
Response: Explained that the folds should be similar but not identical, because they are validated on different cities, and that higher validation loss means worse predictions but should only be compared within the same setup.
Usage: Used to understand and interpret my results.

Prompt: Check my draft and suggest what to remove and what to add.
Response: Pointed out an inaccurate sentence, missing points and note-style wording, without rewriting my text.
Usage: I corrected the paragraph myself.

10. References

[1] Official Cityscapes Dataset: https://www.cityscapes-dataset.com/

[2] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele “The Cityscapes Dataset for Semantic Urban Scene Understanding” (2016): https://www.cityscapes-dataset.com/wordpress/wp-content/data/cordts2016cityscapes.pdf

[3] O. Ronneberger, P. Fischer, and T. Brox “U-Net: Convolutional Networks for Biomedical Image Segmentation” (2015): https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28

[4] Starting point for baseline implementation and development on Cityscapes (2022): https://github.com/kerrgarr/SemanticSegmentationCityscapes

[5] Adrian Rosebrock “Convolutional Neural Networks (CNNs) and Layer Types” (2021): https://pyimagesearch.com/2021/05/14/convolutional-neural-networks-cnns-and-layer-types/

[6] M. Berman, A. Rannen Triki, M. B. Blaschko “The Lovasz-Softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks” (2018): https://arxiv.org/pdf/1705.08790

[7] D. P. Kingma, J. Lei Ba “ADAM: A Method For Stochastic Optimization” (2015): https://arxiv.org/pdf/1412.6980

[8] I. Loshchilov, F. Hutter “Decoupled Weight Decay Regularization ” (2019): https://arxiv.org/pdf/1711.05101

[9] J. Long, E. Shelhamer, T. Darrell “Fully Convolutional Networks for Semantic Segmentation” (2015): https://www.cv-foundation.org/openaccess/content_cvpr_2015/papers/Long_Fully_Convolutional_Networks_2015_CVPR_paper.pdf

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft) “Deep Residual Learning for Image Recognition” (2015): https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

[11] GeeksForGeeks ReLU Activation Function Diagram: https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/

[12] GeeksForGeeks Residual Connection Diagram: https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/

[13] R. Kohavi, “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection”, *IJCAI* (1995): https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf

[14] A. Paszke, A. Chaurasia, S. Kim, E. Culurciello “ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation” (2016): https://arxiv.org/pdf/1606.02147